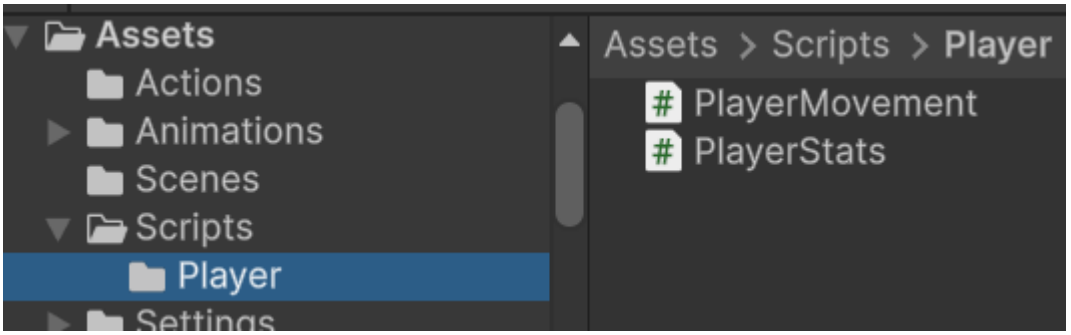
Technical Document - Lab03

Mário Pedro, Rodrigo Roboredo

Level 1

This Lab uses the same project developed in Lab03




```
[CreateAssetMenu(fileName = "PlayerStats", menuName = "PlayerStats")]

Script de Unity | 0 referências
public class PlayerStats : ScriptableObject
```

```
[Header("Config")]
public int playerlever = 1;

[Header("Health")]
public float health = 100;
public float maxhealth = 200;
```




```
public interface IDamagable
{
    1 referência
    public void TakeDamage(float amount);
}
```

```
// O método encontra-se acessível porque está a ser implementado a partir da interface IDamagable
1 referência
public void TakeDamage(float amount)
{

}
```

Level 2

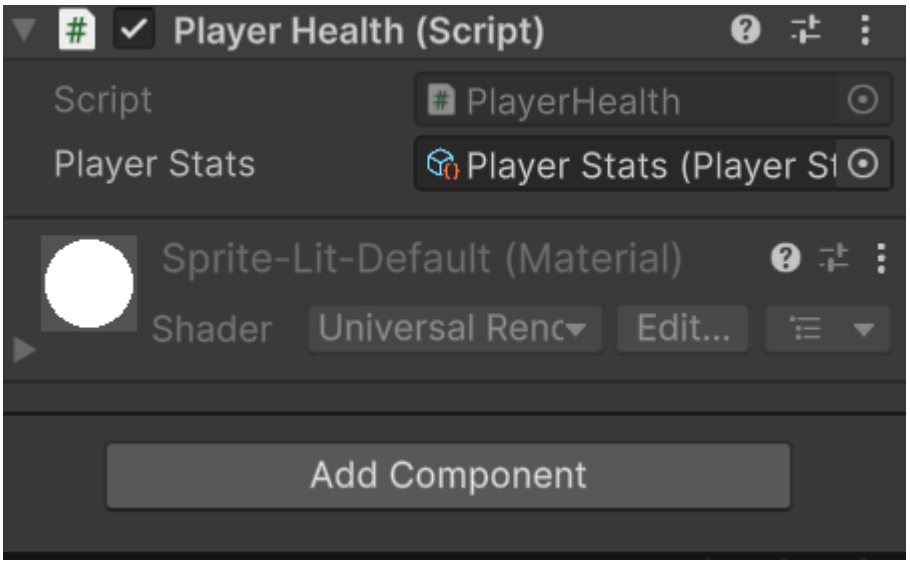
```
[SerializeField]private PlayerStats playerStats;
```

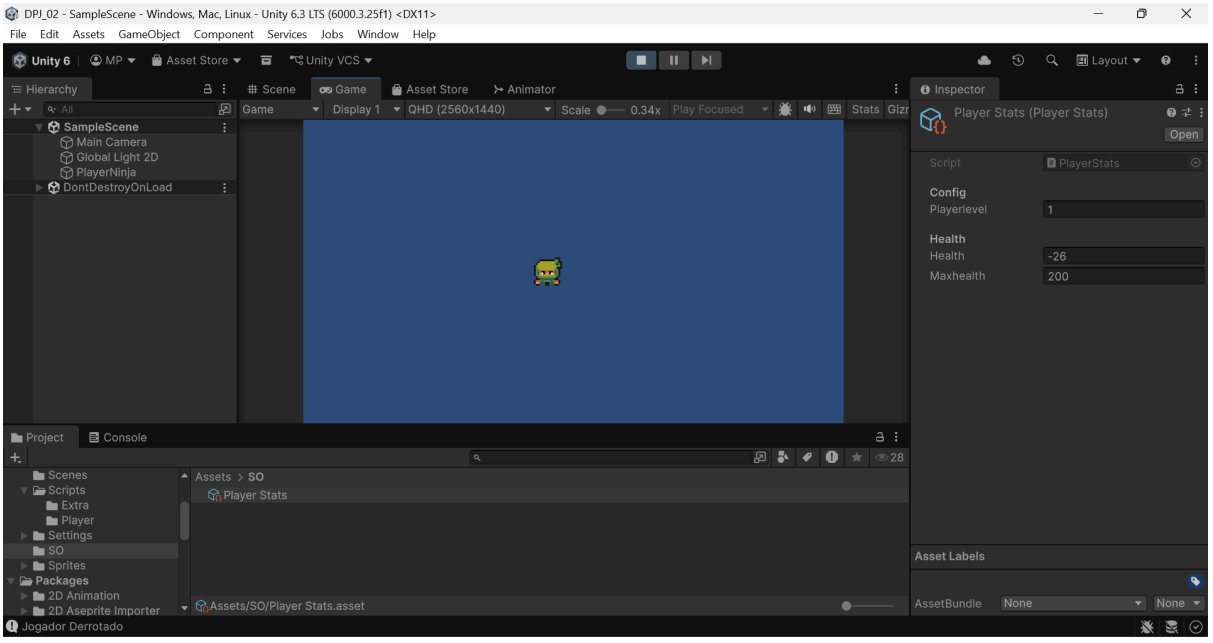
```
public void TakeDamage(float amount)
{
    playerStats.health -= amount;
}
```

```
public void TakeDamage(float amount)
{
    playerStats.health -= amount;
    if (playerStats.health <= 0)
    {
        PlayerDead();
    }
}
```

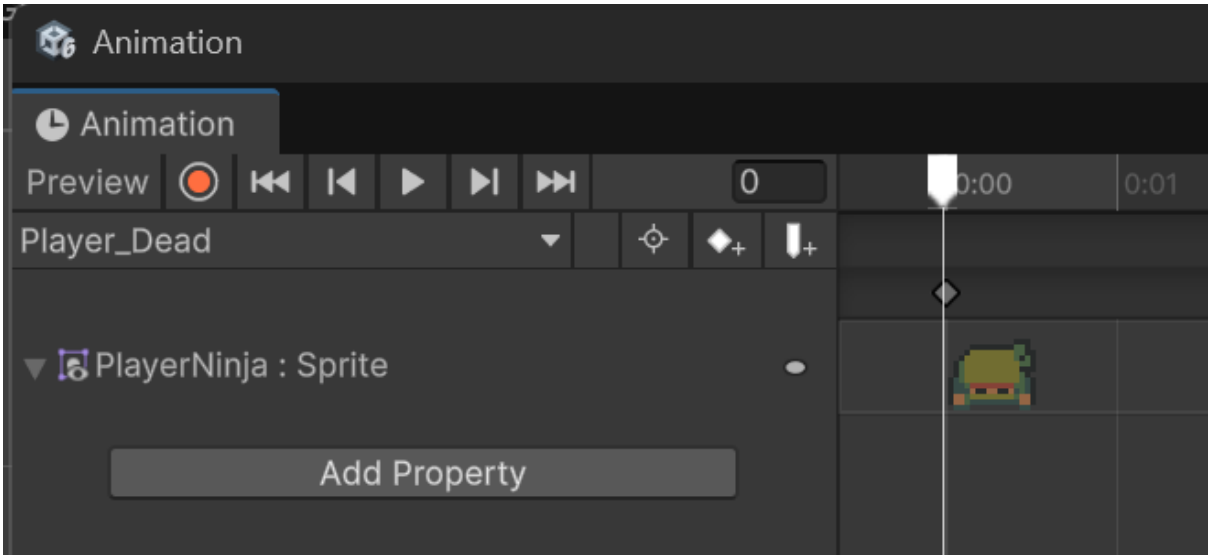
```
public void PlayerDead()
{
    Debug.Log("Jogador Derrotado");
}
```

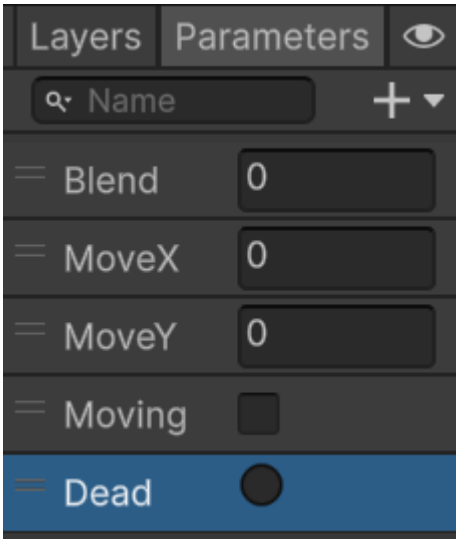
```
void Update()
{
    if (Input.GetKeyDown(KeyCode.P))
    {
        TakeDamage(1f);
    }
}
```

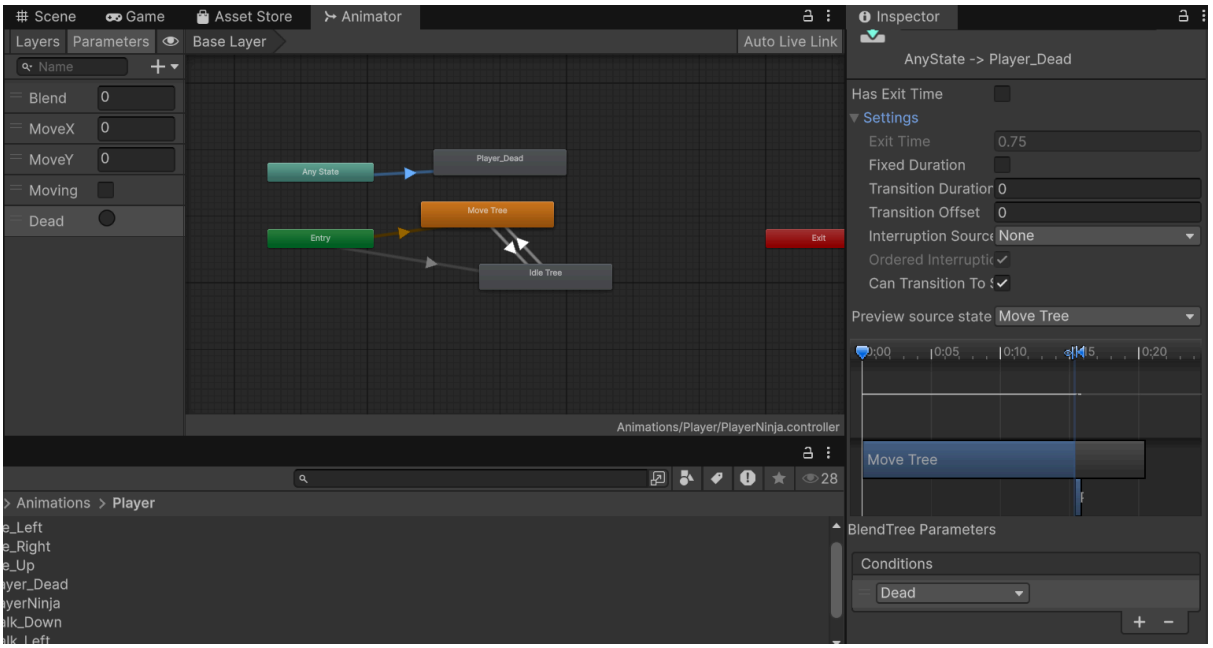




Level 3


Animation
Animation
Preview
0
0:00
0:01
Player_Dead
PlayerNinja : Sprite
Add Property


Layers
Parameters
Name
Blend
0
MoveX
0
MoveY
0
Moving
Dead


Scene
Game
Asset Store
Animator
Layers
Parameters
Base Layer
Auto Live Link
Name
Blend
0
MoveX
0
MoveY
0
Moving
Dead
Any State
Player_Dead
Move Tree
Entry
Exit
Idle Tree
Animations/Player/PlayerNinja.controller
Animations
Player
Inspector
AnyState -> Player_Dead
Has Exit Time
Settings
Exit Time
0.75
Fixed Duration
Transition Duration
0
Transition Offset
0
Interruption Source
None
Ordered Interruptio
Can Transition To S
Preview source state
Move Tree
0:00
0:05
0:10
0:15
0:20
Move Tree
BlendTree Parameters
Conditions
Dead

```
Script de Unity | 0 referências
public class PlayerAnimation : MonoBehaviour
{
    private readonly int moveX = Animator.StringToHash("MoveX");
    private readonly int moveY = Animator.StringToHash("MoveY");
    private readonly int moving = Animator.StringToHash("Moving");
    private readonly int dead = Animator.StringToHash("Dead");
    private Animator animator;

    Mensagem do Unity | 0 referências
    private void Awake()
    {
        animator = GetComponent<Animator>();
    }
```

```
public void SetDeadAnimation()
{
    animator.SetTrigger(dead);
}
```

```
public void SetMoveBoolTransition(bool value)
{

    animator.SetBool(moving, value);

}
```

```
public void SetMoveAnimation()
{
    animator.SetFloat(moveX, dir.x);
    animator.SetFloat(moveY, dir.y);
}
```

Level 4

```
private void ReadMovement()
{
    moveDirection = actions.Movement.Move.ReadValue<Vector2>().normalized;



    if (moveDirection == Vector2.zero)
    {
        playerAnimations.SetMoveBoolTransition(false);

        return;
    }

    playerAnimations.SetMoveBoolTransition(true);
    playerAnimations.SetMoveAnimation(moveDirection);

}
```

```
private void Awake()
{
    playerAnimations = GetComponent<PlayerAnimation>();
}
```

```
public void PlayerDead()
{
    Debug.Log("Jogador Derrotado");
    playerAnimations.SetDeadAnimation();
}

// O método encontra-se acessível porque
2 referências
public void TakeDamage(float amount)
{
    playerStats.health -= amount;
    if (playerStats.health <= 0)
    {
        PlayerDead();
    }
```

Level 5

```
[SerializeField] private PlayerStats playerStats;
0 referências
public PlayerStats Stats => playerStats;
```

```
private void Move()
{
    if (player.Stats.health <= 0)
    {
        return;
    }
```